

Universidade do Estado da Bahia

Bacharelado em Sistemas de



Engenharia de Programas

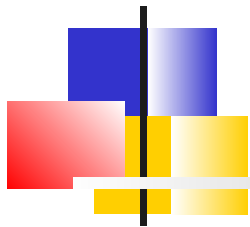
Tema I. Introdução à Engenharia de Programas

Assunto I.1 – Contextualização da Disciplina

Conceitos Básicos de Engenharia

Prof. Diego Frias

diegofrias@uneb.br



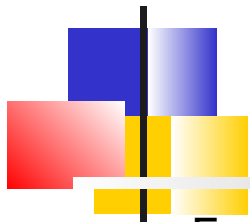
Introdução

A Engenharia de Programas, como seu nome indica trás conceitos e métodos das engenharias para o contexto dos programas de computador

O nome dado à disciplina e seu escopo (ementa) é bem peculiar (não comum) e precisa ser esclarecido logo no início.

Em particular deve ser diferenciado de Engenharia de Software

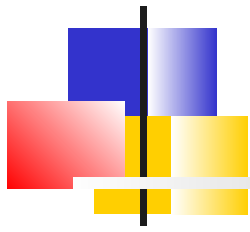
Para isso precisamos diferenciar Programa de Software na concepção da disciplina, descrever o escopo da Engenharia e diferenciar as áreas dela que são abordadas em cada disciplina.



Introdução

- Engenharia (do latim *ingenium*, que significa "inteligência" e *Ingeniare*, que significa "inventar") é a aplicação do conhecimento com vistas a:
 - Inventar (conceber um produto, sistema, dispositivo, etc.)
 - Projetar (detalhar componentes e forma de integração),
 - Construir (desenhar e construir fábricas compostas por máquinas, sistemas, processos e os procedimentos produtivos para garantia da qualidade e a segurança industrial),
 - Operar (definir manuais de procedimentos para operação de instalações de forma eficiente, confiável e segura em regimens normal e de emergência),
 - Manter (definir manuais de procedimentos de monitoração e inspeção e de manutenção preventiva, preditiva e corretiva)

(modificado de Wikipédia – inglês).



5 Princípios Básicos da Engenharia

1. *Viabilidade*
2. *Racionalidade*
3. *Segurança*
4. *Eficiência*
5. *Manutenibilidade*



5 Princípios Básicos da Engenharia

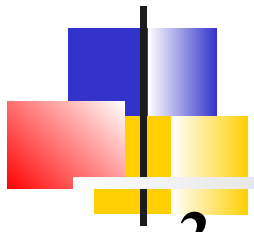
1. ***Viabilidade*** – A decisão de investir ou não no desenvolvimento de um novo produto/material, processo, ou serviço depende dos resultados de 3 análises feitas com 3 pontos de vista diferentes:
- *Viabilidade técnica*: É possível criá-lo com: (a) os materiais, (b) os equipamentos e (c) as pessoas disponíveis/adquiríveis/contratáveis?
 - *Viabilidade econômica*: (a) Estão disponíveis os recursos financeiros necessários para bancar “todos” os custos durante o período de desenvolvimento, de produção, estocagem e comercialização? (b) O tempo até o retorno do investimento é satisfatório?
 - *Viabilidade comercial*: (a) A logística de venda, comercialização e pós-venda foi projetada? (b) O faturamento líquido esperado = $\text{volume_esperado} \times (\text{preço_mercado} - \text{custo_prod} - \text{custo_comerc})$ suporta o tempo de retorno previsto? (c) há um nível razoável de tolerância à reação da concorrência?



5 Princípios Básicos da Engenharia

1. Viabilidade de desenvolvimento de software

- ***Técnica:*** Temos acesso às tecnologias e dispomos dos recursos materiais (computadores de desenvolvimento, teste e operação) e humanos necessários?
- ***Econômica:*** Dispomos dos recursos financeiros necessários para contratação de arquitetos e programadores, para compra/aluguel de computadores e servidores, de licencias de softwares, para aluguel de salas, para despesas correntes – café, água, energia, internet, etc., para marketing, venda e comercialização inicial?
- ***Comercial:*** O produto terá aceitação no mercado e poderá ser vendido na quantidade necessária e no preço necessário para garantir um retorno positivo do investimento num prazo razoável?

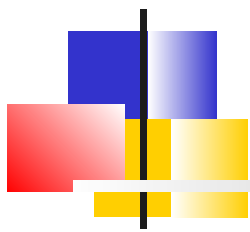


5 Princípios Básicos da Engenharia

2. ***Racionalidade*** – aplicado na fase de design avaliando a relação custo/benefício para a humanidade.

Toda tecnologia envolve um risco mas também traz benefícios (se for viável). Ex. transporte.

Áreas: social (ética e moral), ecológica, ambiental, etc.



5 Princípios Básicos da Engenharia

Racionalidade dos softwares

- *Acreditase que software não causa impacto ambiental (software não polui, não gera rejeitos-lixo, não contamina)*

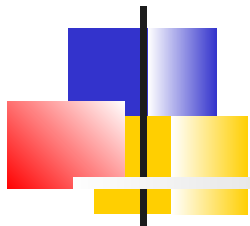
- software somente funciona em algum hardware e o hardware é lixo eletrônico (contaminante com metais pesados).

- Se não existisse software, não existiria hardware, é como corpo e alma! Pode-se culpar somente o corpo?

- *Acreditase que software não introduz **risco** social, não machuca ninguém, não causa mortes.*

- softwares são responsáveis pelo controle automático de todos os dispositivos, equipamentos, sistemas, instalações em todas as indústrias. Então falhas, bugs, malfuncionamento podem conduzir a acidentes industriais (colisão de trens, queda de avião, blecaute, etc.)

- a carência/falha de filtros nas redes sociais podem conduzir a eventos que causem dano moral/psicológico nas pessoas que podem induzir até suicídio.



5 Princípios Básicos da Engenharia

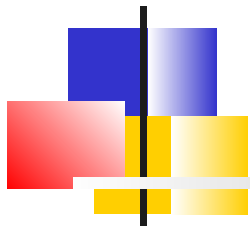
3. ***Segurança*** – aplicado desde o design até a operação e manutenção de indústrias, sistemas e dispositivos.

Este princípio exige minimizar os

Riscos = Danos * Taxa provável de falha/acidente

ficando confinados dentro dos valores “aceitáveis” pela
sociedade?

ALARA - As Low As Reasonably Available.



5 Princípios Básicos da Engenharia

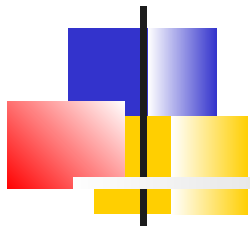
3. *Segurança:*

Danos: (a) material, (b) ecológico, (c) humano e (d) financeiro.

Todos os danos **não-financeiros** induzem um dano **financeiro** derivado.

Exemplo:

- Acidente na torre de petróleo da British Petroleum no mar Caribe
 - dano fin. direto: queda do valor das ações,
 - dano fin. derivado de dano ecológico: custos de recuperação ambiental e multas por dano ambiental,
 - dano fin. derivado de dano material: custos do reparo da torre de petróleo, perda de produção, multas por não entrega, etc.
 - dano fin. derivado de dano humano: indenizações às famílias das vítimas, etc.



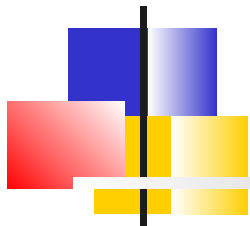
5 Princípios Básicos da Engenharia

3. *Segurança*

Taxa provável de falha/acidente: dada em falha/acidente por ano é “estimada” (chutada) na fase de projeto e vai sendo ajustada com a experiência de uso.

O chute para novas tecnologias (como os veículos autônomos) é mero chute.

Que eu saiba a **sociedade** não tem sido consultada. Pelo menos eu não sei qual é o limiar atual de risco ALARA, nem votei a favor ou em contra.



5 Princípios Básicos da Engenharia

3. Segurança do software

Software pode causar distintos tipos de dano às pessoas por:

1. Mal funcionamento, que conduz à perda ou a distorção de dados, ou à perda de controle de um dispositivo ou instalação por exemplo: sistemas de controle em indústrias de alto risco, de tráfego ferroviário, terrestre, aéreo, de controle de equipamentos médicos e cirúrgicos, etc.
2. Design defetuoso, que permita acesso ilegal aos dados dos usuários, o acesso de alheios às contas bancárias, etc.



5 Princípios Básicos da Engenharia

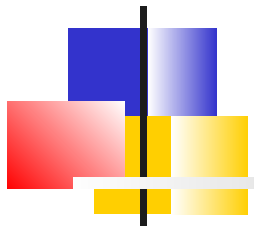
4. ***Eficiência*** – É um princípio que deve ser aplicado desde o design até a operação e manutenção de indústrias, sistemas e dispositivos.

Requer minimizar a relação entre os recursos necessários para:

- Construir - Produzir (na fase de produção), e
- Operar & Manter (na fase de utilização)

e o resultado (benefício, retorno) obtido

$$\eta = \frac{W_{produzido}}{E_{utilizada}} \quad \underline{\text{É o inverso da relação custo/benefício}}$$



5 Princípios Básicos da Engenharia

4. Eficiência

O resultado deve ser medido na unidade apropriada dependendo da fase. Por exemplo, na fase de produção o resultado está relacionado à quantidade de produtos produzidos por unidade de tempo mas, na fase de uso, vai depender do que o produto entrega.

A energia gasta deve ser entendida como investimentos totais, incluindo energia elétrica, insumos, recursos humanos, materiais e financeiros.

Na fase de produção a energia é o somatório dos gastos totais de produção por unidade de tempo.

Na fase de uso, a energia é o que o usuário precisa gastar por unidade de tempo para receber os resultados.



5 Princípios Básicos da Engenharia

4. Eficiência do software

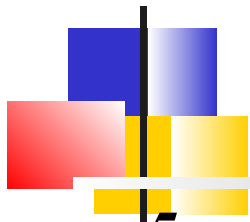
Segundo as fases:

Durante o desenvolvimento: Minimizar os recursos investidos (humanos, materiais e financeiros) para desenvolver as funcionalidades do software

Durante a utilização:

- Minimizar os recursos necessários para executar o software (complexidade-custo do hardware, a largura de banda da rede necessária, o consumo de energia, etc)
- Minimizar o esforço (número de operações) e o tempo necessário para executar as diferentes tarefas do software

NOTA: o consumo de energia é proporcional ao número de operações.



5 Princípios Básicos da Engenharia

5. ***Manutenibilidade*** – Princípio que precisa ser aplicado nas fases de design e projeto.

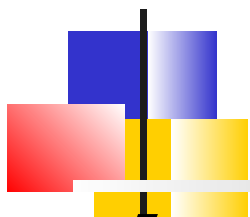
Requer viabilizar, simplificar e minimizar o tempo e os custos das manutenções:

- **Preventiva**

planejada de acordo com o tempo previsto de desgaste de peças e partes, ou não planejada após identificação de falha frequente antecipada de componente ou parte → recall

- **Preditiva** (sugerida por monitoração+predição de momento de falha)

- **Corretiva** (após revisão ou incidente → reparo, conserto)



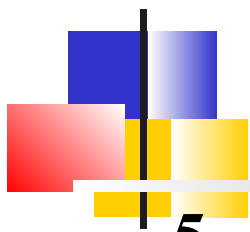
5 Princípios Básicos da Engenharia

5. Manutenibilidade do software

Os softwares não precisam de manutenção preventiva porque não se desgastam pelo uso.

Mas sim de manutenção corretiva. Por exemplo, o desempenho de sistemas de bancos de dados, pode ir degradando-se com o tempo de uso, devido a uma gestão incorreta ou ineficiente do lixo (informação morta, registros sem links, etc).

Os sistemas operacionais começam a ficar lentos pelas mesmas razões, precisando de manutenção periódica para correção de problemas.



5 Princípios Básicos da Engenharia

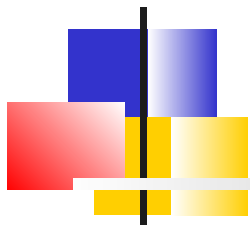
5. Manutenibilidade do software (cont.)

Durante a fase de desenvolvimento a maioria dos sistemas apresentam erros de execução não detectados nos testes alfa. Assim sendo, num período inicial de uso dos sistemas se realiza manutenção corretiva o que é comumente conhecido como correção de bugs (na fase beta).

Princípios de projeto devem ser usados para facilitar a manutenção, como por exemplo, explorar ao máximo a modularidade, o encapsulamento e o reuso de código usando mecanismos de herança e polimorfismos.

Estruturas modularizadas também facilitam a manutenção evolutiva para o desenvolvimento de novas versões.

NOTA: o termo manutenção evolutiva não é usado em outros setores industriais. Apenas na indústria de software.



Software vs Programa

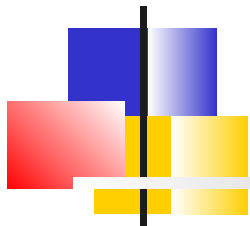
Na nossa disciplina Programa é o “componente material” de um Software.

O Programa é constituído por bits contidos em arquivos binários (quando compilados) ou de texto (quando é código fonte).

É imutável enquanto o arquivo matenha sua integridade e pode ser copiado (clonado) quantas vezes seja necessário (salvo se o arquivo estiver protegido contra cópia).

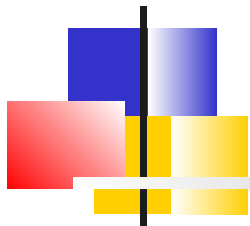
Não qualquer cópia de um Programa é capaz de executar o Software correspondente, exceto se for de uso público.

Nós trabalhamos com Programas contidos em arquivos de texto, ou seja, contendo o código fonte do software.



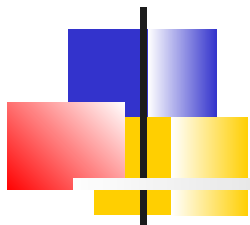
Programa-Algoritmo

1. Todos os programas implementam (executam) 1 ou vários algoritmos.
2. Cada algoritmo, e partes dele, podem ser executados em uma thread simples, ou em várias threads.
3. O algoritmo executado em cada thread define uma sequência de operações sobre dados de diversas natureza gerando dados também de natureza diversa.
4. Em EP fazemos abstração da origem e a forma de captura/acesso aos dados de entrada assim como o destino e a forma de entrega dos dados gerados.
5. O foco será dado ao número e tipo de operações realizadas pelo algoritmo em estudo no processo de transformar dados de entrada em dados de saída.



Algoritmo-Código Fonte

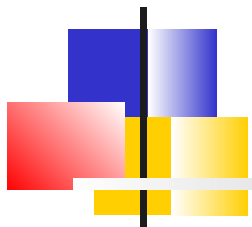
1. Um algoritmo pode ser escrito em distintas linguagens de programação gerando-se código-fonte diferentes.
2. Um algoritmo pode ser escrito de diversas formas usando a mesma linguagem de programação gerando-se código-fonte diferentes.
3. O número e tipo de operações realizados por códigos fonte que implementam um mesmo algoritmo diferem entre si, assim como o uso da memória.
4. O tempo de execução é “proporcional” ao número de operações, sendo também influenciado pelo tipo de operações, pelo tipo (inteiro, booleano, real, caractere) das variáveis envolvidas e pela forma (estrutura) de armazenamento delas.



Objetivo da EP

1. Desenvolver e aplicar métodos que permitam “estimar” o tempo de execução T de códigos fonte baseados em:
 - Técnicas padronizadas de “contagem” do número de operações de diferentes tipos.
 - Quantas operações aritméticas (N_o), quantas atribuições (N_a) e quantas comparações (N_c) são realizadas?
 - Técnicas de calibração de sistemas (SO+hardware)
 - Quanto tempo demora em média realizar uma operação aritmética (T_o), uma atribuição (T_a) e uma comparação (T_c)?

É óbvio que $T = N_o * T_o + N_a * T_a + N_c * T_c$!



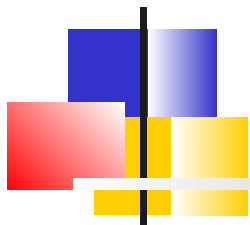
Objetivo da EP

2. Desenvolver e aplicar métodos que permitam “estimar” o requerimento máximo (Mmax) e médio (M) de memória para executar um determinado código fonte baseados em:
- Técnicas padronizadas de “contagem” do número de variáveis de diferentes tipos.
 - Quantas variáveis inteiras (Vi), quantas reais (Vr), quantas booleanas (Vb) e quantas de caracteres (Vc) são instanciadas simultaneamente no código fonte?

A memória média pode ser estimada então da forma

- $M = V_i * M_i + V_r * M_r + V_b * M_b + V_c * M_c$

onde Mi, Mr, Mb e Mc são a memória (bytes) requerida para armazenar 1 inteiro, 1 real, 1 booleano e 1 caractere.

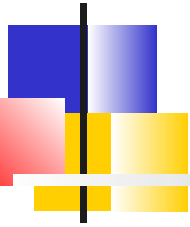


Ementa e Avaliação

Nosso curso centrar-se-á na estimativa do tempo, não da memória.
A ementa e o plano de curso estão no portal acadêmico para consulta.

Além das técnicas de calibração e contagem direta de operações serão vistos 2 métodos “analíticos” para inferir o número de operações em ciclos iterativos complexos e em algoritmos recorrentes, respectivamente.

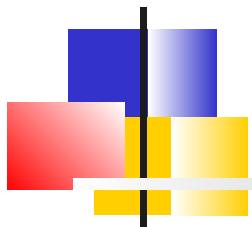
A disciplina contempla de 2 a 3 projetos práticos, em equipes (3 alunos), nos quais os alunos desenvolvem um script Python para resolver problemas simples, mais reais, na área de processamento de imagens.



Ementa e Avaliação

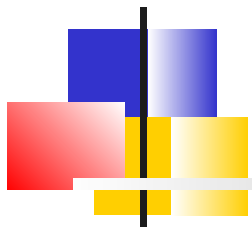
A complexidade de tempo destes algoritmos será medida:

- 1- de **forma experimental** [crédito 1 - daqui a quatro(4) semanas 22.09.2026],
- 2- de **forma analítica** [crédito 2 - daqui a sete(7) semanas 20.10.2026] e
- 3 - de **forma semi-analítica** [crédito 3 - daqui a onze(11) semanas 17.11.2026].



Ementa e Avaliação

A depender do avanço do curso, no terceiro crédito serão apresentados e discutidos técnicas de minimização do tempo de execução de códigos fonte, que não serão objeto de avaliação.



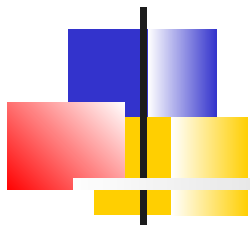
Encontros Presenciais (6 x 2h = 12h)

Durante a realização dos projetos teremos aulas presenciais práticas para acompanhamento e suporte.

A apresentação dos resultados dos projetos será realizada em apresentação gravada pela equipe, onde a participação de todos os integrantes é avaliada de forma individual.

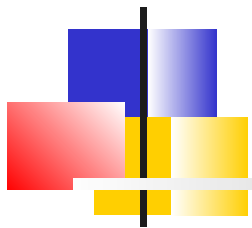
As apresentações serão colocadas para acesso público no Teams.

A critério do professor, algumas apresentações de destaque poderão ser escolhidas para apresentação presencial, também.



Canais de contato

1. Grupo Whatsapp EP_2026.2
(opcional – recomendado) 71-99242-9076
2. Teams: serão lançadas as tarefas e suas avaliações +material de aula (quando houver)
3. E-mail: diegofrias@uneb.br (oficial)
diegofriass@gmail.com (operacional)



FIM